Grid component architecture plan

Component name: `AppGrid` (generic — not tied to any single grid's business purpose)
Core hook: `useGridModel` **Underlying library:** React + AG Grid

This document consolidates the architecture, phase plan, and implementation rules for a single, reusable grid component used across the application. Every grid in the app — regardless of business purpose — is an instance of `AppGrid` configured differently, not a separate component.

---

1. Core principle

One component. One public hook. Many configs.

`AppGrid` never knows what business data it's displaying. It only knows how to interpret a config object. What changes between a risk grid, a trade blotter, or a counterparty exposure grid is the **config passed in** — never the component code.

```
.proto response → Adapter (parses to JSON) → gridData { rows, columns, metadata }
```

Service config vs. UI config — the dividing line

Owned by service/proto config	Owned by UI-side code
Column list, field names, order	Which React component renders a given `renderAs` value
Data type per column (`text` , `currency` , `percentage` , `date` , `enum`)	Icon choice, exact colors, spacing, density
Which renderer to use (`renderAs: "trend"`)	The renderer implementations themselves
Row identity field, row severity value	How severity maps to a color/icon
Feature toggles (sortable, filterable, paginated)	Theme tokens, fonts, animations
Master-detail flag + detail reference	Layout of the detail panel shell

Rule of thumb: if changing it requires backend/data team involvement, it's service config. If it's purely visual, it's UI-side, driven by a semantic hint the service provides (not a raw color or icon name).

---

2. Component API — target shape

Responsibility boundary: the consuming application owns all service and proto concerns — calling APIs, parsing proto responses into JSON, and assembling the `gridData` payload. The component (`AppGrid`) never talks to services, never sees proto types, and never knows where the data came from. It receives one prop, `gridData` , and internally transforms it into everything AG Grid needs: `rowData` , `columnDefs` , `defaultColDef` , **and** `gridOptions` .

```
App layer:        service call → proto → parsed to JSON → gridData
                                                            |
                                                            ▼
Component layer:  <AppGrid gridData={...} onRowClick={...} />
                                                            |
                                                            ▼
Internal to AppGrid:  gridData.rows      → rowData
                      gridData.columns   → columnDefs
                      gridData.columns   → defaultColDef (shared defaults)
                      gridData.metadata → gridOptions (pagination,
                                           row selection, density, etc.)
```

```
<AppGrid
  gridData={{
    rows: normalizedRows,        // parsed from proto → JSON, app-side
    columns: resolvedColumns,    // column schema, from the same payload
    metadata: {                  // grid-level info the app assembled
      gridId: "portfolio-var",
      mode: "client",
      rowIdField: "id",
      pagination: { pageSize: 50 }
    }
  }}
  onRowClick={handleRowClick}
  loading={isLoading}
  className="page-specific-layout"
/>
```

- `gridData` — the single data-shape prop. `AppGrid` derives AG Grid's `rowData` , `columnDefs` , `defaultColDef` , and `gridOptions` from `gridData.rows` ,

`gridData.columns`, and `gridData.metadata` respectively, via the internal normalization layer. The app never constructs raw AG Grid props by hand, and never passes a `gridOptions` object directly.

- `onRowClick`, `onCellClick`, and similar — passed as plain top-level callback props from the app. Internally, `AppGrid` wires these into the AG Grid `gridOptions` it builds (e.g., `onRowClicked`), so the app never needs to know AG Grid's own callback names.
- `loading`, `className` — genuine UI-lifecycle/layout props that belong to the consuming page.

What does NOT become a prop: column shape, feature toggles, renderer choice, row severity logic, pagination mode, row selection behavior — all of that lives inside `gridData`, resolved internally by `AppGrid`. The app never authors `columnDefs` or `gridOptions` directly.

---

3. Config contract

`gridData` is the JSON payload the service layer produces after parsing the proto response — this is what actually arrives at the component, already shaped into `rows` / `columns` / `metadata`:

```
gridData = {
  rows: [ /* parsed, normalized row objects */ ],
  columns: [
    {
      field: "diffPct",
      headerName: "Diff",
      dataType: "percentage",          // drives formatting
      renderAs: "trend",               // resolved via renderer registry
      sortable: true,
      filter: "range",
      pinned: null
    }
    // ...
  ],
  metadata: {
    gridId: "string",
    mode: "client" | "server",         // pagination mode
    hasDetail: boolean,
    rowIdField: "id",
    pagination: { pageSize: 50 }
  }
```

```
}
```

A **normalization layer** (`normalizeGridData`) turns `gridData` into AG Grid's actual `rowData` / `columnDefs` / `defaultColDef` shape, applying defaults and validation in one place. This is the seam that absorbs:

- AG Grid version changes (property renames between versions)
- Proto/service field naming differences
- Missing/invalid config (defaults + validation, not silent failure)

Even where service field names already match AG Grid's own `ColDef` naming (a deliberate convenience, not a hard coupling), this seam stays in place — cheap when names align, essential the day they don't.

---

4. Proto integration

Proto messages are **not passed into the component or the UI layer at all**. The flow is:

```
.proto response → parsed to plain JSON (service/adapter layer)
                → normalized into gridData { rows, columns, metadata }
                → passed to <AppGrid gridData={...} />
```

`AppGrid` and everything inside it (`useGridModel` , the renderer registry, `normalizeGridData`) only ever operates on plain JSON. Proto types never cross into the component layer — this is a hard boundary, not just a convention.

- The **parsing step** (proto → JSON) is owned by the service/adapter code, one adapter per grid/message type (`adapters/<gridName>Adapter.js`). Adapters are the only code in the codebase aware of proto field names or generated proto types.
- Adapters are **defensive by default** — every proto field access is optional (`protoRow.getField?.() ?? default`) so additive proto changes never break existing grids silently, and the resulting JSON always has a predictable shape even if a field is missing.
- Business-derived fields (e.g., `severity`) are computed **once, during parsing**, and included directly in the row JSON — not recalculated inside cell renderers on every render.
- Because the component boundary is plain JSON, the same `gridData` shape works identically regardless of the backend transport — proto today, a REST/JSON API or

websocket feed tomorrow — without touching `AppGrid`, the renderer registry, or `useGridModel`.

- A lightweight parser/adapter test per grid type guards against silent drift when a `.proto` file changes and the parsing step isn't updated to match — this is where schema drift should surface, not inside the UI.

---

5. Phase plan

Phase 1 — Core grid *(done)*

Strict internal contract (normalized data + feature toggles + event handlers). AG Grid invocation kept minimal in the view layer.

Phase 1B — Service contract boundary *(done)*

`gridData` shape (`rows` / `columns` / `metadata`) defined as the single prop contract. Normalization layer maps `gridData` → AG Grid's internal `rowData` / `columnDefs` / `defaultColDef`, with defaults and validation.

Phase 2 — Sorting & filtering

- Sort/filter behavior fully config-driven (column-level `sortable`, `filter` type, default sort model).
- Filter UI (chips, dropdowns) chosen by UI layer based on the filter *type* the config declares — service says the type, UI owns the widget.

Phase 3 — Cell renderer registry

- Renderer **selection** is config-driven (`renderAs: "trend"`); renderer **implementation** lives in a central UI-side registry.
- Direction (up/down) is inferred safely from value sign — no config needed for this.
- Severity/coloring comes from an explicit `severity` value already present on the row (computed in the adapter or by the service) — the renderer paints it, it does not guess business meaning from a raw number.

Phase 4 — Icon & visual styling layer

- Icon selection is a UI-side lookup keyed by semantic values the data carries (`severity: "danger"` → warning icon) — services never send icon names or colors.

- One shared icon/theme registry used by every grid instance app-wide.

Phase 5 — Server-side pagination

- Config declares `mode: "server"` ; sort/filter state is sent as request parameters instead of applied client-side.
- Uses AG Grid's Server-Side Row Model with a `getRows` -style datasource wired to the service layer.
- Handled as an internal branch inside `useGridModel` 's composition, not a separate hook or component fork.

Phase 6 — Master-detail grids

- Config declares `hasDetail: true` plus a detail schema/endpoint reference.
- Detail panel is rendered as another `AppGrid` instance with its own config — recursive use of the same pipeline, not special-cased code.
- Deferred combination with server-side pagination until Phase 5 is stable, since the two together are the highest-complexity combination.

Phase 7 — Interaction polish

Column pin/resize/reorder with persistence, row expand/collapse, keyboard navigation, right-click actions. Density toggle exposed in the UI, not just available as config.

Phase 8 — Performance & re-render control

- Immutable data mode + stable `getRowId` .
- `columnDefs` / `gridOptions` / `defaultColDef` / `rowData` all memoized inside `useGridModel` , recomputed only when `gridData` actually changes.
- All cell renderer components memoized (`React.memo`) — the most common source of perceived slowness with custom renderers at scale.
- Debounced filter/search input before triggering grid updates.

Phase 9 — Loading & skeleton states

- Dedicated skeleton component mirroring real column widths and expected row count — not a generic spinner.
- Skeleton variants account for grid mode (initial load vs. per-page load for server-side grids).

- Subtle top-bar progress indicator for background refresh, distinct empty vs. error states with a retry action where relevant.

Phase 10 — Scalability & governance

- Config schema validation at the normalization boundary (reject unknown `renderAs` values, missing required fields, malformed types) — catches bad config before it reaches AG Grid.
- Config versioning on the service contract, with a deprecation path for breaking changes.
- Error boundary around each `AppGrid` instance so one misconfigured grid doesn't take down the page.
- Lightweight telemetry hook (slow grids, malformed configs, deprecated feature usage) built in early rather than retrofitted later.
- Accessibility as an explicit checklist: keyboard navigation, ARIA roles for grid/row/cell, screen-reader labeling — not an afterthought.
- A migration path (adapter or codemod) for teams moving existing AG Grid usage onto `AppGrid` incrementally.

Phase 11 — Modern visual language *(cross-cutting, applied from Phase 3 onward)*

- Soft, layered surfaces; rows read as a structured grid (hairline row dividers, bounded container) rather than isolated floating cards.
- Inline data visualization in cells: trend arrows, utilization bars, sparklines — not just raw numbers.
- Smooth, purposeful micro-interactions (row hover, density transitions, expand/collapse) — not instant, jarring state changes.
- Skeleton loaders with subtle shimmer in place of spinners; friendly empty states.
- Dark mode treated as first-class across every renderer, icon, and color token from the start, not tested in after the fact.
- Icons: outline-style icon set (Tabler/Lucide/Phosphor) for a current, non-skeuomorphic look. Fonts: a neutral UI sans (Inter/IBM Plex Sans) for labels, a monospace face (JetBrains Mono/IBM Plex Mono) for numeric columns.

Sequencing note: treat Phase 11 as a lens applied throughout Phases 3–9, not a final visual pass — retrofitting modern styling onto an already-dense grid is expensive; building it in

from the renderer phase onward is not. Prioritize Phase 8 (performance) before expanding visual scope further — a fast, plain grid is a better foundation than a feature-rich, sluggish one.

---

6. Internal hook composition

`useGridModel` is a single public hook, composed internally from small, single-purpose hooks — not one large function with branching logic:

```
useGridModel (public API)
  ├── useGridCore        (columns, rowData, base state)
  ├── useSorting
  ├── useFiltering
  ├── usePagination       (branches client/server internally)
  ├── useMasterDetail     (active only when config.hasDetail)
  └── useDensityTheme
```

This keeps the public surface stable (`useGridModel(gridData, otherProps)`) while allowing new grid capabilities to be added as new internal hooks, without risking unrelated functionality when one area changes.

Component implementation pattern

`AppGrid` itself stays a thin wrapper: it calls `useGridModel` , gets back everything AG Grid needs, applies the density theme class, and renders `AgGridReact` . It never touches `gridData` fields directly — that's the hook's job.

```
function AppGrid({ gridData, ...otherProps }) {
  const { columnDefs, rowData, defaultColDef, gridOptions, densityMode } =
    useGridModel(gridData, otherProps);

  const themeClassName = getGridThemeClassName(densityMode);

  return (
    <div className={themeClassName} style={{ ...GRID_THEME_STYLE, ...GRID_CONTAIN
      <AgGridReact
        rowData={rowData}
        columnDefs={columnDefs}
        defaultColDef={defaultColDef}
        gridOptions={gridOptions}
      />
    </div>
```

```
  );
}

export default AppGrid;
```

- `gridData` (rows/columns/metadata) and `otherProps` (callbacks like `onRowClick`, layout props like `className`) go into `useGridModel` together.
- `useGridModel` is where `gridData` is normalized into `rowData` / `columnDefs` / `defaultColDef`, where `otherProps` callbacks are wired into AG Grid's native `gridOptions` callback names, and where `densityMode` is resolved from metadata.
- `AppGrid` itself contains no transformation logic — it only destructures the hook's output and renders. This is what keeps the component thin even as grid capability grows across later phases.

---

7. Naming and coupling discipline

- Field names in service/proto config may deliberately match AG Grid's own `ColDef` naming for convenience, but the service layer never imports or depends on AG Grid types directly — it stays a plain data contract.
- Any AG Grid property that is inherently a function (`valueFormatter`, `cellRenderer`, `comparator`) is never sent as config directly — the service sends a semantic key (`renderAs`, `dataType`), and the UI registry resolves the real implementation.
- The normalization layer remains the single seam absorbing naming drift between proto, service config, and whatever version of AG Grid is in use.

---

8. What stays specific to each grid instance (by design)

Not everything should be generic — these remain legitimately per-instance:

- The actual `gridData` (rows/columns/metadata) for that grid's data.
- Adapter logic mapping that grid's specific proto message.
- Business-specific severity computation (what counts as "bad" for this particular data) — computed in the adapter, not the shared UI layer.
- Event handler callbacks (`onRowClick`, etc.) passed as plain props by the consuming

page — wired into AG Grid's `gridOptions` internally by `AppGrid`, never authored by the app directly.

Everything else — the component, the hook composition, the renderer registry, the icon/theme system, the skeleton loader, the normalization layer — is shared and generic across every grid in the application.